

Technical Approach

Companion to the concept document (product) and the diagrams in `diagrams/` (system architecture, frontend architecture, ERD, state diagram). This document is the source for the "Technical Approach" section of the concept presentation and written proposal.

Frontend architecture

Vite + React + TypeScript + MUI, themed to Greenstone's actual brand (see the design-system notes in the repository). Routing via React Router; client/session state in Redux Toolkit; server state in TanStack Query — see the frontend architecture diagram (see appendix) for the full reasoning and the exact folder structure, which matches this repo's `src/` directly rather than being aspirational.

Why Vite over Next.js for this build. The JD lists both across Greenstone's products. Atlas Marketplace is an authenticated, behind-login application (GPs and LPs, not public search-engine traffic) — SSR's main win, first-paint SEO, doesn't apply to most of the surface. A public marketing/discovery layer (e.g. an unauthenticated "browse open raises" page for SEO) would be the concrete trigger to introduce Next.js for that slice specifically, while keeping the authenticated app as a Vite SPA — not an all-or-nothing framework choice.

Data model and API

Full entity shapes and relationships: the data-model diagram (see appendix). Core REST surface (Laravel, versioned under `/api/v1`):

Endpoint	Purpose
<code>GET /fund-vehicles</code>	List/filter open raises (strategy, geography, vintage)
<code>GET /fund-vehicles/{id}</code>	Fund detail — narrative, track record, benchmark series
<code>POST /fund-vehicles</code>	GP creates a listing (<code>DRAFT</code>)
<code>PATCH /fund-vehicles/{id}</code>	GP edits a listing; blocked once <code>OPEN</code> on fields that would invalidate submitted indications (amount terms, hard cap) — see trade-offs
<code>POST /fund-vehicles/{id}/publish</code>	<code>DRAFT</code> → <code>OPEN</code> , requires track-record + data-room docs present
<code>GET /fund-vehicles/{id}/indications</code>	GP's view of submitted indications (own fund only, RBAC-enforced)
<code>POST /fund-vehicles/{id}/indications</code>	LP submits an indication — idempotency key required, server computes allocation (see state diagram)

Endpoint	Purpose
PATCH /indications/{id}	Withdraw (LP) or decline-with-reason (GP)
GET /organizations/{id}	Org profile — track record, accreditation status
GET /fund-vehicles/{id}/data-room	Signed, time-limited S3 URLs per document
GET/POST /fund-vehicles/{id}/messages	Deal-scoped message thread
GET /me/notifications	Polled fallback for clients not on the WebSocket connection

The frontend consumes these through TanStack Query hooks in each feature's `queries.ts` (pattern established in `features/fund-vehicles/queries.ts`) — mutations invalidate rather than optimistically write server-decided fields (allocation amounts, committed totals), for the reasons in the state diagram's concurrency section.

Real-time and transaction states

Full lifecycle and the concurrency mechanism: the state diagram (see appendix). In production, live "% subscribed" and allocation notifications are delivered over Pusher private/presence channels scoped per fund vehicle, authenticated through a Laravel Echo-style auth endpoint (so an LP can only subscribe to channels for funds they're permitted to see). The frontend's `useSubmitIndication` mutation invalidates the relevant queries on both its own success and on an incoming WebSocket event for that fund — two paths into the same reconciliation, so a client sees a consistent number whether the update came from its own action or another LP's.

Security

- **Auth:** Entra ID SSO (the JD names Microsoft 365/Entra ID explicitly — Greenstone users likely already have Entra accounts) for staff/Admin, with Laravel Sanctum-issued tokens for the API session; external GP/LP users authenticate the same way but through a separate, more tightly scoped client credential.
- **RBAC:** four roles (LP , GP , ADMIN , COMPLIANCE) enforced server-side on every endpoint, not just hidden in the UI — a GP's `GET /fund-vehicles/{id}/indications` only ever returns rows for funds that GP's org owns.
- **Sensitive data:** KYC documents and financial identifiers encrypted at rest (RDS column-level encryption for identifiers, S3 SSE for documents), signed/expiring S3 URLs for data-room access rather than public links, full audit log on every `IndicationOfInterest` state transition (who, when, from what state) for Compliance review and dispute resolution.

- **OWASP Top 10 practices:** CSRF tokens on state-changing requests, Tiptap output sanitized server-side before storage and render (stored-XSS is the realistic risk for any rich-text listing narrative), rate-limiting on POST /indications (ElastiCache-backed) to prevent a scripted flood of submissions from gaming an oversubscribed allocation.
- **DIFC data protection:** framed throughout rather than bolted on — data residency for AWS resources, explicit consent/purpose fields on KYC data collection, and the audit-log-by-default design above exist because of this, not despite it.

What the prototype implements today

The prototype enforces the same model against a mock API, so the behaviour can be demonstrated rather than described: mock sign-in (staff through a mock Entra ID chooser, external users by e-mail and password), 30-minute sliding sessions, logout after five failed attempts, and one permissions table (role × organization accreditation × ownership) that both refuses requests in the API (401/403/409/422) and explains disabled actions in the UI. Tenant scoping, accreditation approval by an admin, and an append-only audit trail are implemented and covered by tests. Deliberately not production-grade: the session token sits in `sessionStorage` (production uses an `httpOnly`, `Secure`, `SameSite` cookie via Sanctum), and there is no real identity provider, MFA or password hashing.

Performance and quality

Route-level code splitting per feature area (planned for production — the prototype ships a single bundle); MUI X Data Grid's built-in virtualization for the GP investor pipeline and LP watchlist (both genuinely data-dense views); Sentry for frontend error and performance monitoring; WCAG AA as the accessibility floor (keyboard navigation, focus management in dialogs, color contrast — see the design-system notes in the repository's measured ratios). Testing strategy is deliberately scoped, not exhaustive: Vitest + RTL unit tests concentrated on the state-machine/allocation logic (`src/mock/api.test.ts` is the working example — the part of this system most likely to hide a real bug), component tests for the two or three components with actual conditional logic, and one Playwright E2E over the critical path (browse → submit indication → see status update). Scoping where test effort goes, rather than chasing coverage percentage, is itself one of the judgement calls this brief asks to see.

Delivery

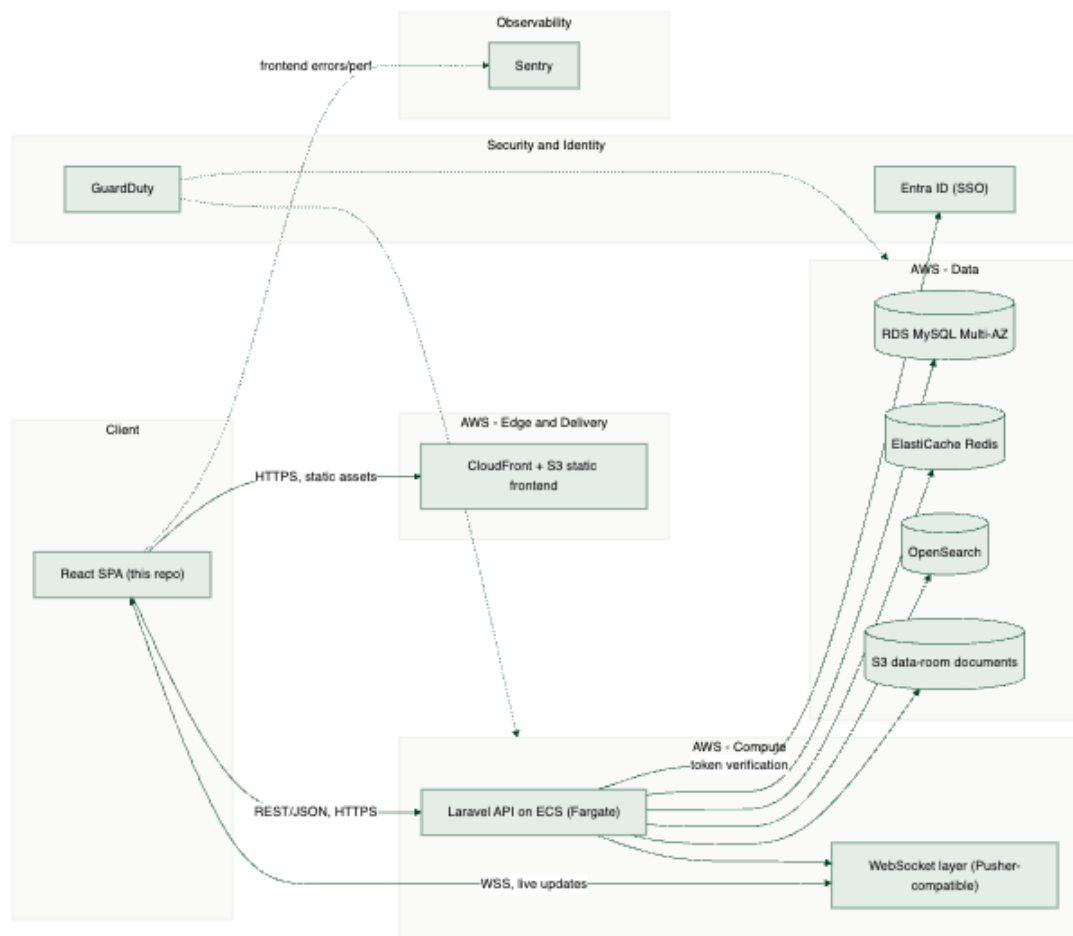
Frontend: static build to S3 + CloudFront in production (this prototype deploys to Vercel instead, purely for a fast, zero-infra demo link — not the production recommendation). API: Laravel on ECS Fargate behind an ALB. CI/CD via GitHub Actions — lint + typecheck + unit tests on every PR, a preview deploy per PR for design/product review, promotion to staging on merge to `main`, and a manual-approval gate to production. Three environments (dev/staging/prod) on separate AWS accounts to keep production data genuinely isolated, given the sensitivity of what's stored here.

Trade-offs and what's deferred

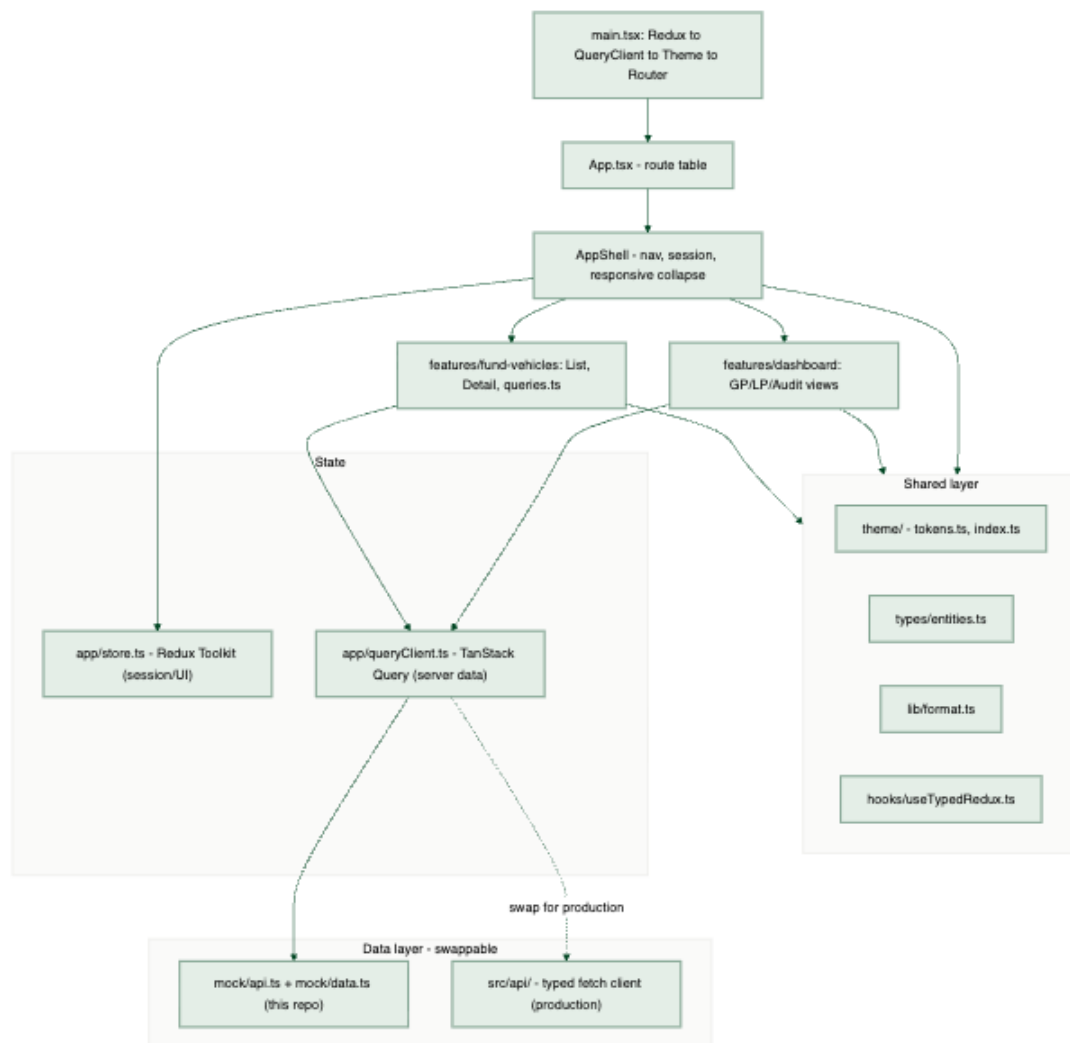
Decision	Alternative considered	Why this way	Revisit when
Redux + TanStack Query split	One library for both (e.g. Redux alone with RTK Query)	Keeps server-state caching/invalidation semantics (staleness, refetch-on-window-focus) separate from genuinely client-only state; matches the JD's explicit "Redux, TanStack Query" pairing	If the app ever needs offline-first writes, RTK Query's mutation queue might justify consolidating
Vite SPA	Next.js SSR	No public SEO surface for the authenticated core product	A public discovery/marketing layer is added
Client-side fund filtering	OpenSearch from day one	v1 listing volume doesn't need it; ships faster	Listings exceed a few hundred and filter latency becomes visible
Mocked KYC/AML status field	Real KYC/AML vendor integration	Vendor selection is a compliance/procurement decision, not a frontend architecture one	Before any real fund closes on the platform
Pro-rata scale-back as the default allocation policy	Pure priority queue (first-submitted wins)	Matches how real fundraises are actually allocated; a GP can override with a logged reason	Not planned to change — this is a considered choice, not a placeholder
In-platform commitment tracking only, no escrow/payment rails	Full payment processing on-platform	Regulatory and integration scope far exceeds a v1; funds already have administrators for capital calls	If Greenstone wants to disintermediate fund administrators entirely — a much later, much bigger decision

Appendix: diagrams

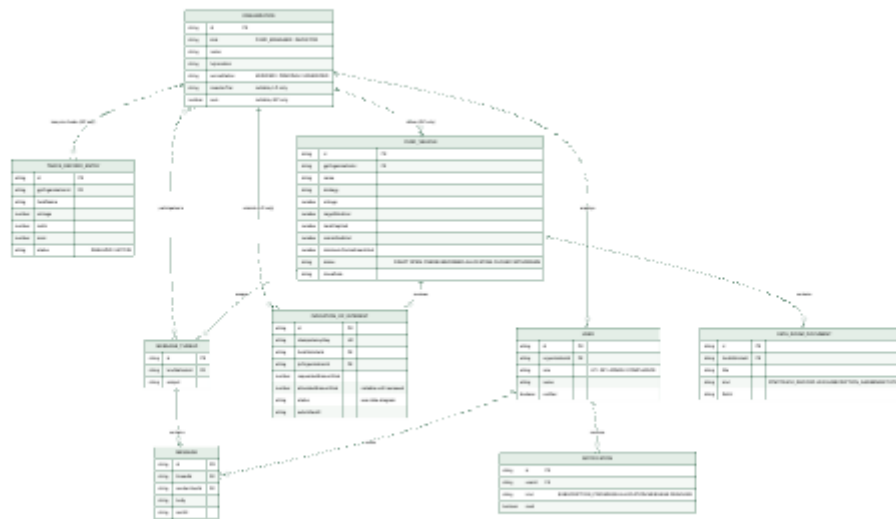
System architecture



Frontend architecture



Data model (ERD)



Indication state diagram

